data is available in our serial buffer. If so, it reads the first byte and, so long as the newline escape code has not been sent, will check to see if that value of the byte represents a digit from 0 to 9. This sketch uses the Newline option in the Serial Monitor, so that each time we hit the Enter key, the Serial Monitor sends the escape code `'/n'` for a new line. We can test for this in our `if` statement like any other value to know when all of the digits for our angle have been entered.

At this point, the code will put together our angle integer digit-by-digit or byte-by-byte using a little math to convert the ASCII character into its numerical value, and then increases the angle value by a power of ten for each byte received. Once it receives the newline character or `'/n'` from the Serial Monitor, it will move on to check the angle value that was received to make sure it falls within the capabilities of the servo. If so, it moves the servo to that angle and sends that angle value to the Serial Monitor. If not, it will send a message letting us know to try again. Now let's look at how this is done.

Serial Library

The Serial library is a little different from the other libraries that are a part of the standard Arduino distribution. First, the library is automatically included in our sketches along with the normal Arduino library without us needing to do anything. Second, an instance of the Serial library is already created for us in the background that, on the Arduino Uno, is called `Serial`, and on the Arduino Mega might be `Serial1`, `Serial2`, or `Serial3`. So when we use one of the upcoming functions, it will be preceded by the instance name `Serial` and a period to separate the library name from the procedural or function name. As an aside, you might have noticed by now that many of the different libraries have similar procedures, including `begin()`, `print()`, `write()`, and others, so it is important that we get the instance of the library correct or who knows what will happen! Enough posturing, let's move onward with our first Serial function.

begin()

With the Serial library already included for us and an instance called `Serial` already created, the first thing we need to do to work with the Serial functions is to establish a data transfer speed. To do that we use the `begin()` function.

```
Serial.begin(datarate)
```

Again, our instance name on the Arduino Uno is `Serial`, so we will stick with that for our instance name. The `datarate` parameter is the speed measured in bits per second that, for communicating with our computer, should be one of the following speeds: 300, 1200, 2400, 4800, 9600, 14400, 19200, 28800, 38400, 57600, or 115200. Generally, you'll find that 9600 is a fairly common speed and is more than fast enough to do what we want to do. In our project sketch, this function took the following form:

```
Serial.begin(9600);
```

This sets up our communication to occur at a speed of 9600 bps. You might have a reason to set a more esoteric speed if your device calls for it, but whatever speed is set, both devices need to be set for the same speed. To do this in the built-in Serial Monitor, there is a drop-down menu in the bottom-right corner of the window where these speeds can be selected.